

## Week 2: EDA and Preprocessing

In [33]: `!pip install ucimlrepo`

Requirement already satisfied: ucimlrepo in c:\users\rushw\anaconda3\lib\site-packages (0.0.7)  
 Requirement already satisfied: pandas>=1.0.0 in c:\users\rushw\anaconda3\lib\site-packages (from ucimlrepo) (2.3.3)  
 Requirement already satisfied: certifi>=2020.12.5 in c:\users\rushw\anaconda3\lib\site-packages (from ucimlrepo) (2025.11.12)  
 Requirement already satisfied: numpy>=1.26.0 in c:\users\rushw\anaconda3\lib\site-packages (from pandas>=1.0.0->ucimlrepo) (2.3.5)  
 Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\rushw\anaconda3\lib\site-packages (from pandas>=1.0.0->ucimlrepo) (2.9.0.post0)  
 Requirement already satisfied: pytz>=2020.1 in c:\users\rushw\anaconda3\lib\site-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)  
 Requirement already satisfied: tzdata>=2022.7 in c:\users\rushw\anaconda3\lib\site-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)  
 Requirement already satisfied: six>=1.5 in c:\users\rushw\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0)

In [34]: `import pandas as pd  
 import numpy as np  
 import matplotlib.pyplot as plt  
 import seaborn as sns  
 from sklearn.model_selection import train_test_split  
 from sklearn.preprocessing import StandardScaler`

In [35]: `from ucimlrepo import fetch_ucirepo`

```
heart_disease = fetch_ucirepo(id=45)

X = heart_disease.data.features
y = heart_disease.data.targets

df = pd.concat([X, y], axis=1)
df.shape
```

Out[35]: (303, 14)

In [36]: `df.head()`

Out[36]:

|   | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca  | th  |
|---|-----|-----|----|----------|------|-----|---------|---------|-------|---------|-------|-----|-----|
| 0 | 63  | 1   | 1  | 145      | 233  | 1   | 2       | 150     | 0     | 2.3     | 3     | 0.0 | 6.0 |
| 1 | 67  | 1   | 4  | 160      | 286  | 0   | 2       | 108     | 1     | 1.5     | 2     | 3.0 | 3.0 |
| 2 | 67  | 1   | 4  | 120      | 229  | 0   | 2       | 129     | 1     | 2.6     | 2     | 2.0 | 7.0 |
| 3 | 37  | 1   | 3  | 130      | 250  | 0   | 0       | 187     | 0     | 3.5     | 3     | 0.0 | 3.0 |
| 4 | 41  | 0   | 2  | 130      | 204  | 0   | 2       | 172     | 0     | 1.4     | 1     | 0.0 | 3.0 |

# 1. Dataset Overview

This dataset is the Heart Disease dataset from the UCI Machine Learning Repository (Cleveland database) and it is used for a binary classification task.

The goal is to predict the `num` column, where `1` means the patient has heart disease and `0` means the patient does not.

The dataset contains 303 rows and 14 columns, with 13 input features and 1 target column.

The features include both numerical (age, trestbps, chol, thalach, oldpeak) and categorical (sex, cp, fbs, restecg, exang, slope, ca, thal) variables, which describe patient demographics and clinical test results.

```
In [37]: df.dtypes
```

```
Out[37]: age          int64
sex          int64
cp          int64
trestbps    int64
chol        int64
fbs         int64
restecg     int64
thalach     int64
exang       int64
oldpeak     float64
slope       int64
ca          float64
thal        float64
num         int64
dtype: object
```

```
In [38]: df.isnull().sum()
```

```
Out[38]: age          0
sex          0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
num         0
dtype: int64
```

```
In [39]: df.describe()
```

Out[39]:

|              | age        | sex        | cp         | trestbps   | chol       | fbs        | restecg    |
|--------------|------------|------------|------------|------------|------------|------------|------------|
| <b>count</b> | 303.000000 | 303.000000 | 303.000000 | 303.000000 | 303.000000 | 303.000000 | 303.000000 |
| <b>mean</b>  | 54.438944  | 0.679868   | 3.158416   | 131.689769 | 246.693069 | 0.148515   | 0.990000   |
| <b>std</b>   | 9.038662   | 0.467299   | 0.960126   | 17.599748  | 51.776918  | 0.356198   | 0.994000   |
| <b>min</b>   | 29.000000  | 0.000000   | 1.000000   | 94.000000  | 126.000000 | 0.000000   | 0.000000   |
| <b>25%</b>   | 48.000000  | 0.000000   | 3.000000   | 120.000000 | 211.000000 | 0.000000   | 0.000000   |
| <b>50%</b>   | 56.000000  | 1.000000   | 3.000000   | 130.000000 | 241.000000 | 0.000000   | 1.000000   |
| <b>75%</b>   | 61.000000  | 1.000000   | 4.000000   | 140.000000 | 275.000000 | 0.000000   | 2.000000   |
| <b>max</b>   | 77.000000  | 1.000000   | 4.000000   | 200.000000 | 564.000000 | 1.000000   | 2.000000   |

In [40]: `df["num"].value_counts()`

Out[40]:

```
num
0    164
1     55
2     36
3     35
4     13
Name: count, dtype: int64
```

In [41]: `df["num"].value_counts(normalize=True)`

Out[41]:

```
num
0    0.541254
1    0.181518
2    0.118812
3    0.115512
4    0.042904
Name: proportion, dtype: float64
```

## Initial Summary

The dataset contains a mix of numerical and categorical columns.

The target column `num` confirms that this is a classification problem with two classes.

`isnull().sum()` shows that `ca` and `thal` have missing values, which will need to be handled before modelling.

All other columns have no missing values.

## 2. Missing Value Handling

```
In [42]: print("Missing values per column:")
print(df.isnull().sum()[df.isnull().sum() > 0])
print()
print("ca missing:", df["ca"].isnull().sum())
print("thal missing:", df["thal"].isnull().sum())
```

Missing values per column:

ca 4

thal 2

dtype: int64

ca missing: 4

thal missing: 2

## Missing Value Strategy

We identified missing values in `ca` and `thal`.

For `ca`, we use median because this is an ordinal numerical feature with a small number of missing entries.

For `thal`, we use mode because this is a categorical feature with a limited set of discrete values.

We chose not to drop these rows because the dataset is relatively small (303 rows), and removing them would reduce the available training data.

```
In [43]: df_clean = df.copy()

df_clean["ca"] = df_clean["ca"].fillna(df_clean["ca"].median())

df_clean["thal"] = df_clean["thal"].fillna(df_clean["thal"].mode()[0])

df_clean.isnull().sum()
```

```
Out[43]: age      0
sex        0
cp         0
trestbps   0
chol       0
fbs        0
restecg    0
thalach    0
exang      0
oldpeak    0
slope      0
ca         0
thal       0
num        0
dtype: int64
```

```
In [44]: df_clean.describe()
```

Out[44]:

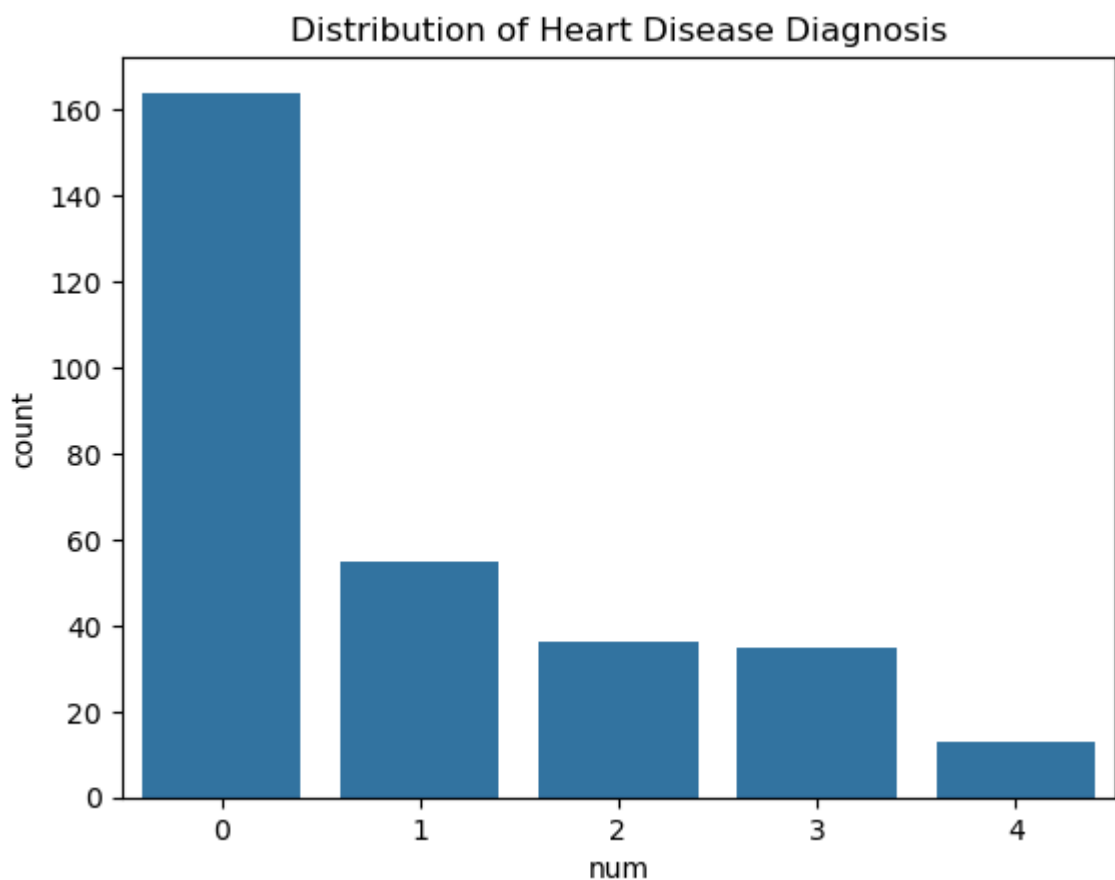
|       | age        | sex        | cp         | trestbps   | chol       | fbs        | restecg    |
|-------|------------|------------|------------|------------|------------|------------|------------|
| count | 303.000000 | 303.000000 | 303.000000 | 303.000000 | 303.000000 | 303.000000 | 303.000000 |
| mean  | 54.438944  | 0.679868   | 3.158416   | 131.689769 | 246.693069 | 0.148515   | 0.990000   |
| std   | 9.038662   | 0.467299   | 0.960126   | 17.599748  | 51.776918  | 0.356198   | 0.994500   |
| min   | 29.000000  | 0.000000   | 1.000000   | 94.000000  | 126.000000 | 0.000000   | 0.000000   |
| 25%   | 48.000000  | 0.000000   | 3.000000   | 120.000000 | 211.000000 | 0.000000   | 0.000000   |
| 50%   | 56.000000  | 1.000000   | 3.000000   | 130.000000 | 241.000000 | 0.000000   | 1.000000   |
| 75%   | 61.000000  | 1.000000   | 4.000000   | 140.000000 | 275.000000 | 0.000000   | 2.000000   |
| max   | 77.000000  | 1.000000   | 4.000000   | 200.000000 | 564.000000 | 1.000000   | 2.000000   |



### 3. Exploratory Data Analysis

#### Relationship Between Features

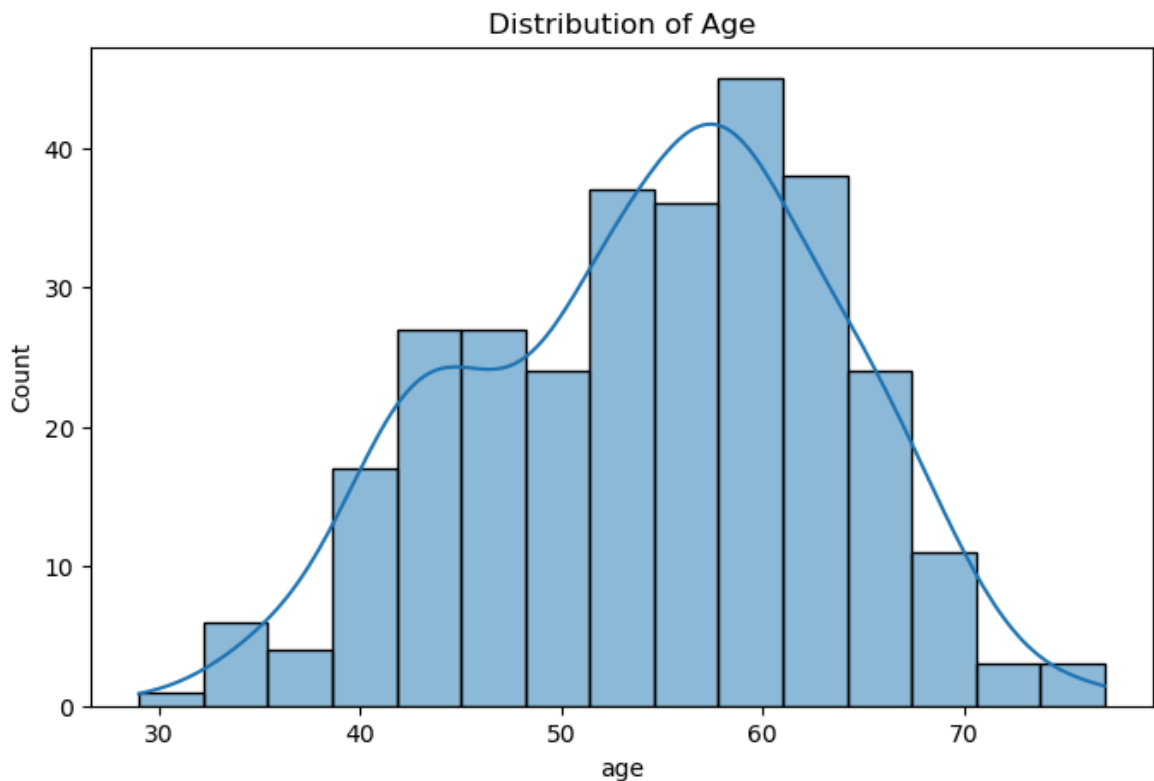
```
In [45]: sns.countplot(x="num", data=df_clean)
plt.title("Distribution of Heart Disease Diagnosis")
plt.show()
```



#### Interpretation of Target Distribution

The target variable is binary and shows the distribution of patients with and without heart disease. The classes are relatively balanced, which is a positive sign for model training because it means the model will not be biased toward the majority class. However, we should still monitor performance using metrics beyond simple accuracy.

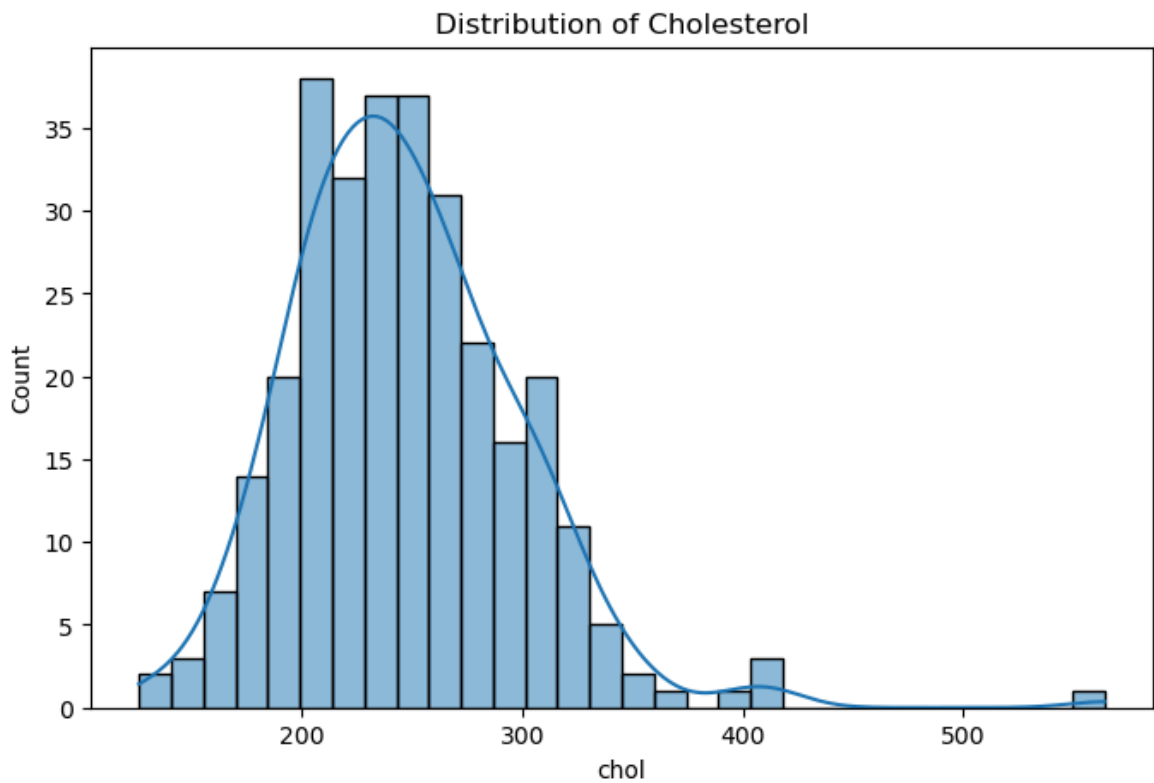
```
In [46]: plt.figure(figsize=(8, 5))
sns.histplot(df_clean["age"], bins=15, kde=True)
plt.title("Distribution of Age")
plt.show()
```



## Interpretation of Age Distribution

The distribution of age is approximately normal, centred around the mid-50s. Most patients in the dataset are between 40 and 65 years old. Since heart disease risk is known to increase with age, this feature is likely to be an important predictor in the model.

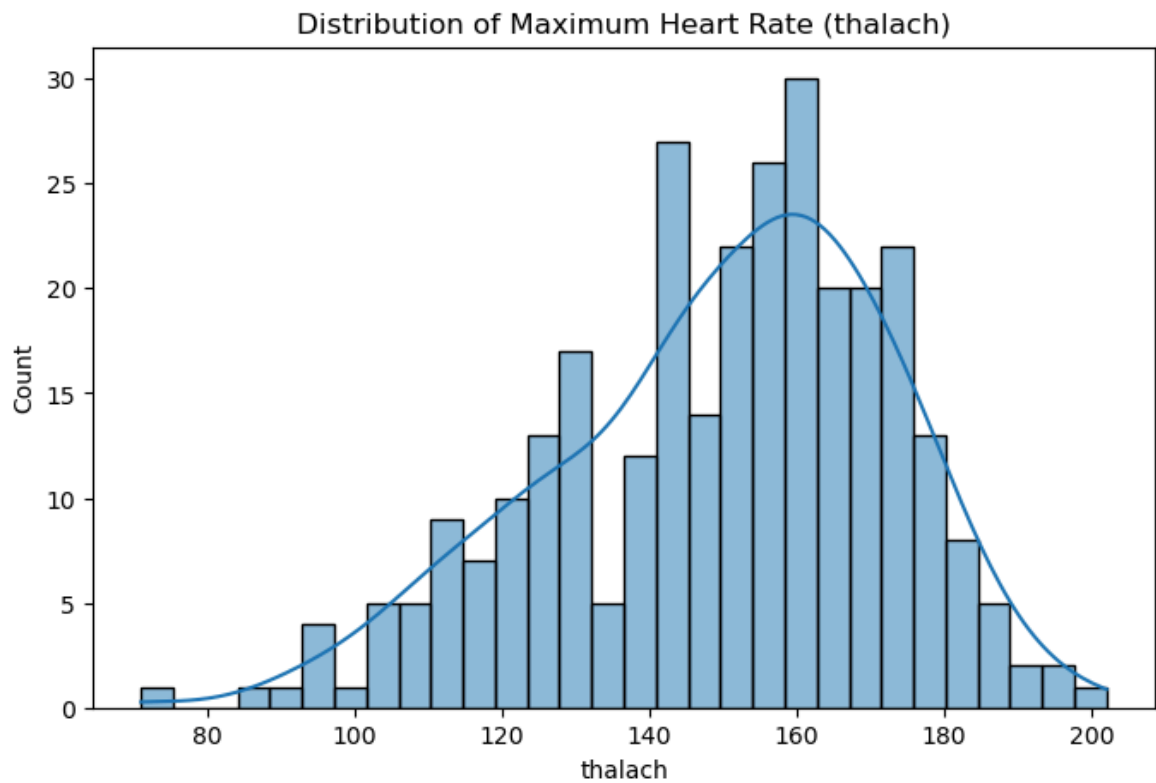
```
In [47]: plt.figure(figsize=(8, 5))
sns.histplot(df_clean["chol"], bins=30, kde=True)
plt.title("Distribution of Cholesterol")
plt.show()
```



## Interpretation of Cholesterol Distribution

Cholesterol values are concentrated in the 200–300 mg/dl range, with some extreme values above 400. High cholesterol is a well-known risk factor for cardiovascular disease, so this feature may help distinguish between healthy and at-risk patients.

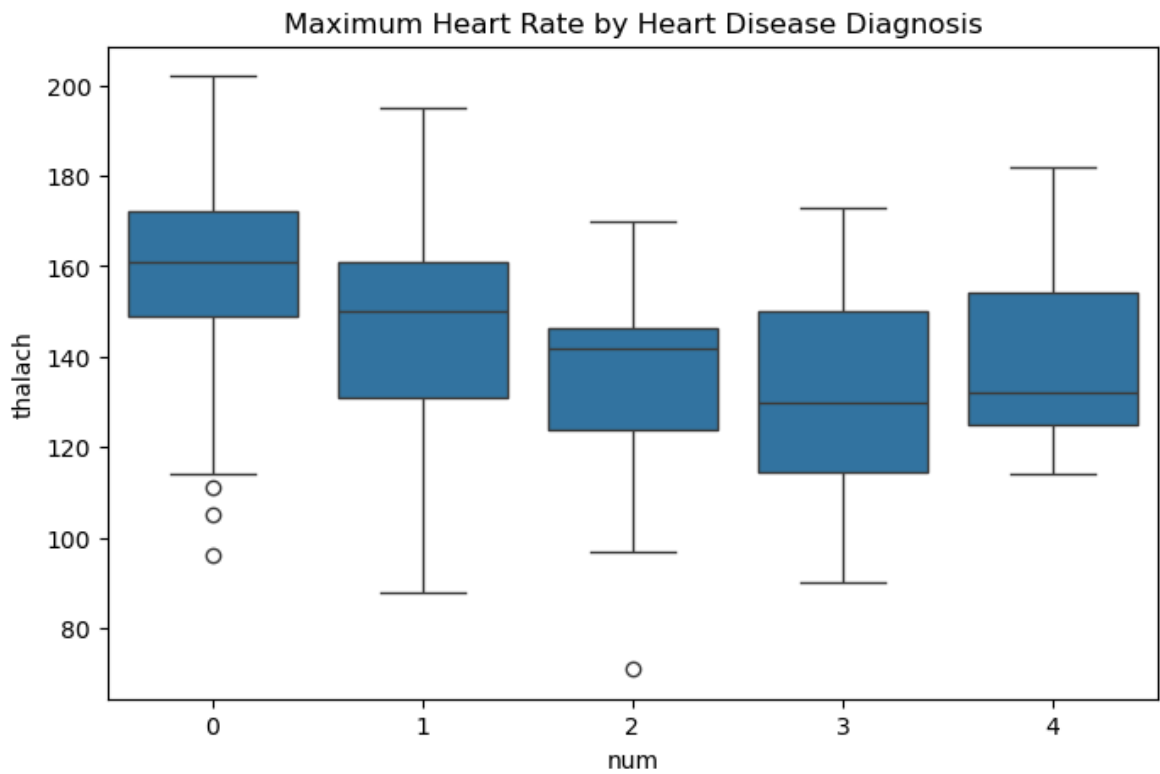
```
In [48]: plt.figure(figsize=(8, 5))
sns.histplot(df_clean["thalach"], bins=30, kde=True)
plt.title("Distribution of Maximum Heart Rate (thalach)")
plt.show()
```



## Interpretation of Maximum Heart Rate Distribution

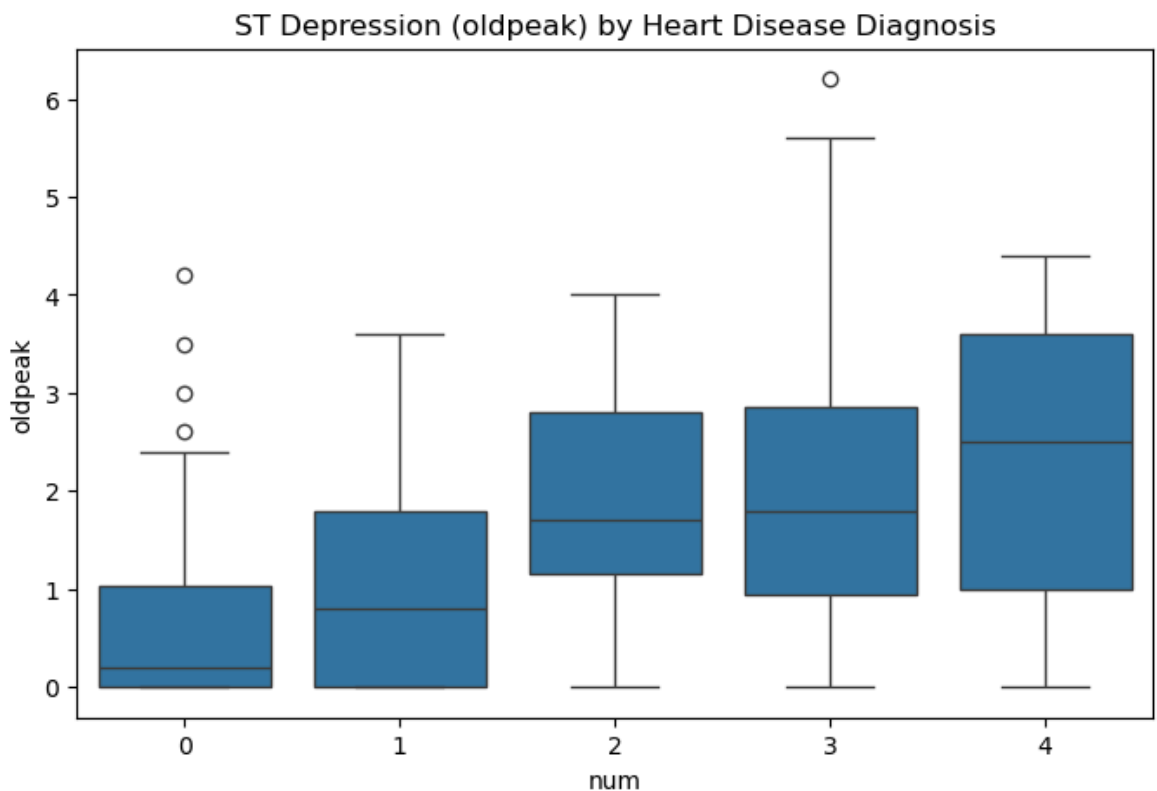
Maximum heart rate achieved is distributed across a wide range, with most values between 120 and 180. This feature is relevant because a lower maximum heart rate during exercise testing may indicate reduced cardiac function, which could be linked to the presence of heart disease.

```
In [49]: plt.figure(figsize=(8, 5))
sns.boxplot(x="num", y="thalach", data=df_clean)
plt.title("Maximum Heart Rate by Heart Disease Diagnosis")
plt.show()
```



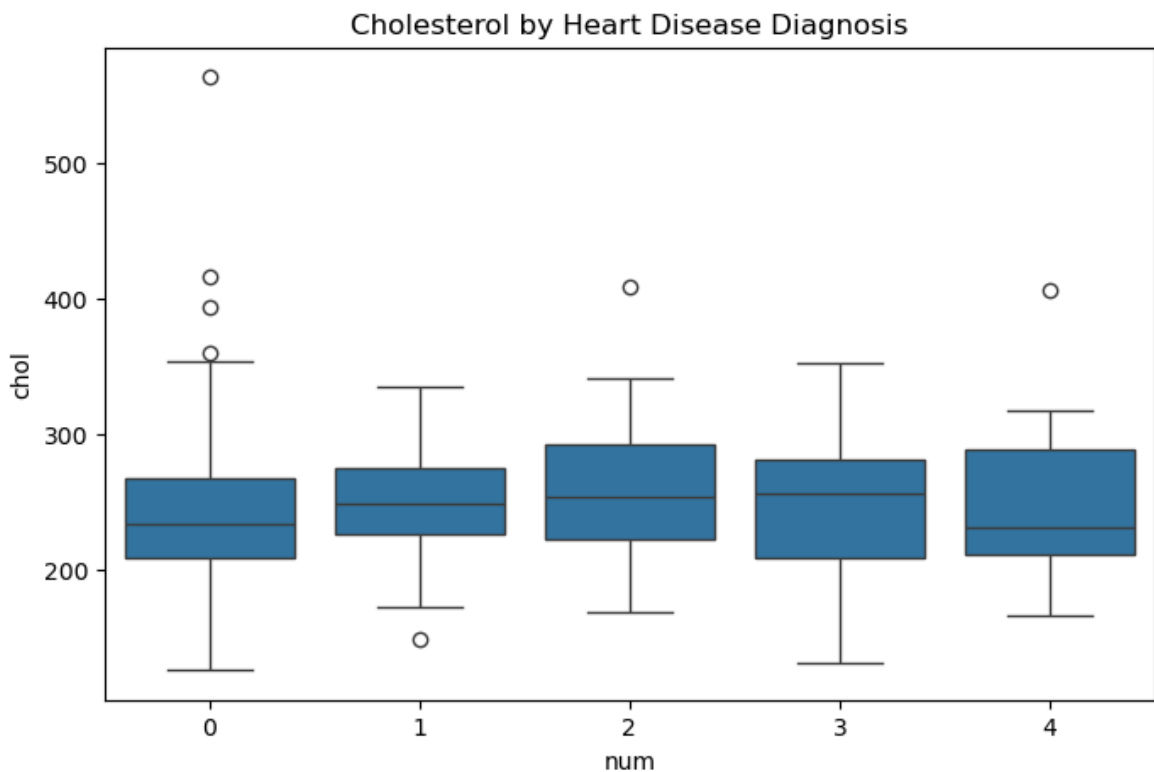
The median maximum heart rate is noticeably lower in the heart disease group. The entire distribution is shifted downward. This suggests that `thalach` is one of the stronger predictors of heart disease in this dataset.

```
In [50]: plt.figure(figsize=(8, 5))
sns.boxplot(x="num", y="oldpeak", data=df_clean)
plt.title("ST Depression (oldpeak) by Heart Disease Diagnosis")
plt.show()
```



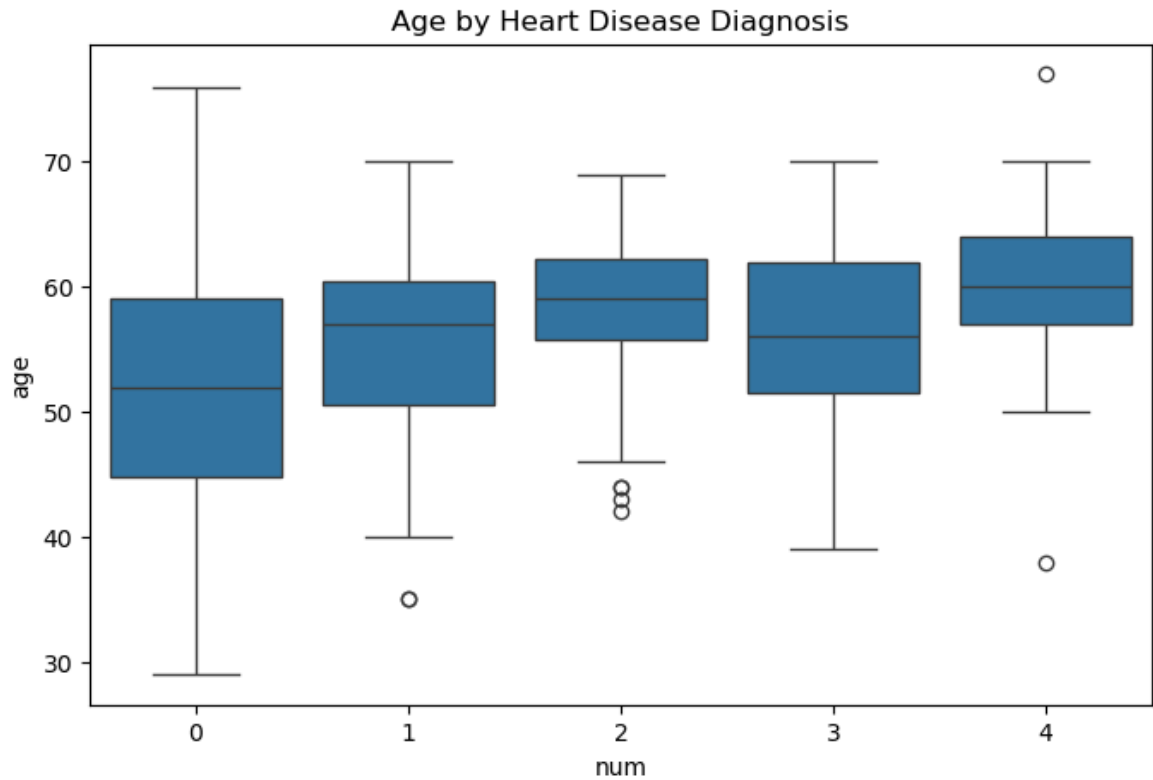
The boxplot shows that patients with heart disease ( `num = 1` ) tend to have higher oldpeak values than patients without heart disease ( `num = 0` ). The median oldpeak is clearly higher in the disease group. This makes clinical sense because ST depression during exercise is a known indicator of myocardial ischemia.

```
In [51]: plt.figure(figsize=(8, 5))
sns.boxplot(x="num", y="chol", data=df_clean)
plt.title("Cholesterol by Heart Disease Diagnosis")
plt.show()
```



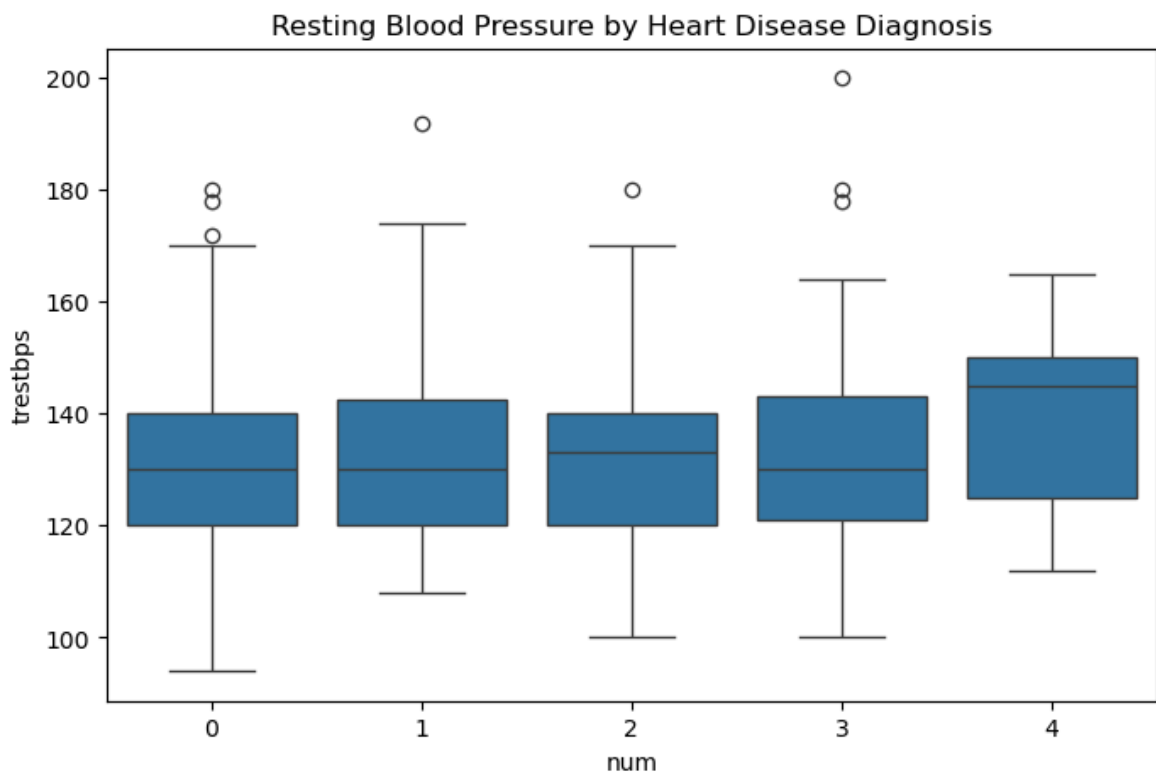
The cholesterol boxplot shows that both groups have similar median cholesterol levels and overlapping distributions. While high cholesterol is a general risk factor for cardiovascular disease, it does not appear to strongly separate the two classes on its own in this dataset.

```
In [52]: plt.figure(figsize=(8, 5))
sns.boxplot(x="num", y="age", data=df_clean)
plt.title("Age by Heart Disease Diagnosis")
plt.show()
```



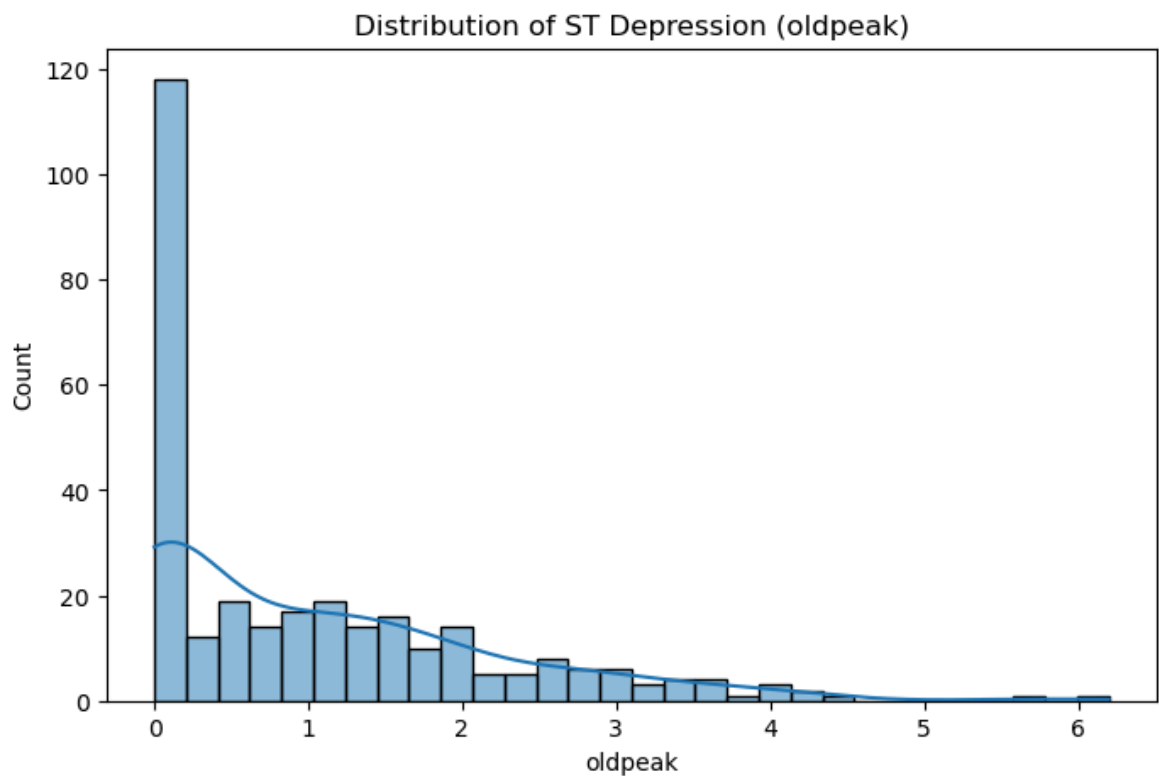
The age boxplot indicates that patients with heart disease tend to be slightly older than patients without it. The median age of the `num = 1` group is higher. Although the groups overlap, age still appears to be a useful feature for prediction.

```
In [53]: plt.figure(figsize=(8, 5))
sns.boxplot(x="num", y="trestbps", data=df_clean)
plt.title("Resting Blood Pressure by Heart Disease Diagnosis")
plt.show()
```



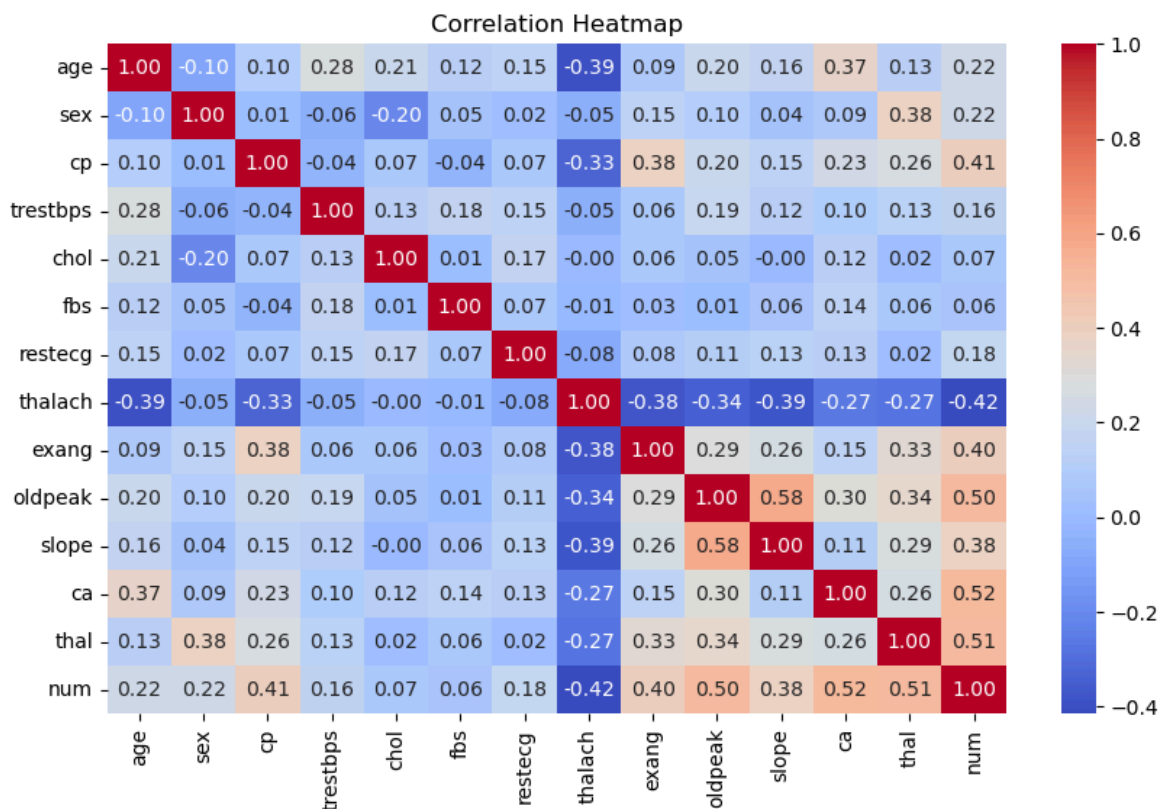
The boxplot shows that patients with heart disease tend to have slightly higher resting blood pressure than patients without. However, the difference between the two groups is relatively small, and there is strong overlap between their distributions.

```
In [62]: plt.figure(figsize=(8, 5))
sns.histplot(df_clean["oldpeak"], bins=30, kde=True)
plt.title("Distribution of ST Depression (oldpeak)")
plt.show()
```



The distribution of `oldpeak` is strongly right-skewed. Most observations are concentrated at lower values close to zero, while a smaller number of patients have much higher ST depression values. This suggests that most patients have minimal exercise-induced ST depression, but those who do may be at higher risk.

```
In [55]: plt.figure(figsize=(10, 6))
sns.heatmap(df_clean.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```



The heatmap shows that `exang` (exercise-induced angina), `oldpeak`, and `cp` (chest pain type) have noticeable positive correlations with the target `num`, which suggests they are likely important predictors of heart disease. `thalach` has a negative correlation with `num`, meaning patients with lower maximum heart rates are more likely to have heart disease. Some feature-to-feature correlations are also visible, such as `age` with `thalach` and `exang` with `oldpeak`, which helps us better understand the structure of the dataset.

## 4. Feature Engineering

```
In [56]: cat_cols = ["cp", "restecg", "slope", "thal"]

df_clean = pd.get_dummies(df_clean, columns=cat_cols, drop_first=True)

df_clean.head()
```

```
Out[56]:
```

|   | age | sex | trestbps | chol | fbs | thalach | exang | oldpeak | ca  | num | cp_2  | cp_3  | cp_4  |
|---|-----|-----|----------|------|-----|---------|-------|---------|-----|-----|-------|-------|-------|
| 0 | 63  | 1   | 145      | 233  | 1   | 150     | 0     | 2.3     | 0.0 | 0   | False | False | False |
| 1 | 67  | 1   | 160      | 286  | 0   | 108     | 1     | 1.5     | 3.0 | 2   | False | False | True  |
| 2 | 67  | 1   | 120      | 229  | 0   | 129     | 1     | 2.6     | 2.0 | 1   | False | False | True  |
| 3 | 37  | 1   | 130      | 250  | 0   | 187     | 0     | 3.5     | 0.0 | 0   | False | True  | False |
| 4 | 41  | 0   | 130      | 204  | 0   | 172     | 0     | 1.4     | 0.0 | 0   | True  | False | False |

## Encoding Rationale

One-hot encoding was applied to `cp`, `restecg`, `slope`, and `thal` because these are nominal categorical features where the numerical values do not represent a meaningful order. For example, chest pain type `0` is not "less than" type `3` — each value represents a different category. One-hot encoding creates separate binary columns for each category, which prevents the model from assuming an incorrect ordinal relationship. We used `drop_first=True` to avoid multicollinearity. The remaining features (`sex`, `fbs`, `exang`) are already binary and do not require encoding.

```
In [63]: df_clean["age_oldpeak"] = df_clean["age"] * df_clean["oldpeak"]
df_clean.head()
```

```
Out[63]:
```

|   | age | sex | trestbps | chol | fbs | thalach | exang | oldpeak | ca  | num | cp_2  | cp_3  | cp_4  |
|---|-----|-----|----------|------|-----|---------|-------|---------|-----|-----|-------|-------|-------|
| 0 | 63  | 1   | 145      | 233  | 1   | 150     | 0     | 2.3     | 0.0 | 0   | False | False | False |
| 1 | 67  | 1   | 160      | 286  | 0   | 108     | 1     | 1.5     | 3.0 | 2   | False | False | True  |
| 2 | 67  | 1   | 120      | 229  | 0   | 129     | 1     | 2.6     | 2.0 | 1   | False | False | True  |
| 3 | 37  | 1   | 130      | 250  | 0   | 187     | 0     | 3.5     | 0.0 | 0   | False | True  | False |
| 4 | 41  | 0   | 130      | 204  | 0   | 172     | 0     | 1.4     | 0.0 | 0   | True  | False | False |

## Feature Engineering Rationale

A new interaction feature, `age_oldpeak`, was created to capture the combined effect of age and exercise-induced ST depression. This was done because older patients with higher ST depression may face a compounded risk of heart disease. This feature may help the model detect patterns that are not fully visible when using age and oldpeak separately.

## 5. Preprocessing

```
In [58]: X = df_clean.drop("num", axis=1)
y = df_clean["num"]
```

## 6. Train-Test Split

At this stage, the dataset is divided into training and testing parts using an 80/20 ratio. The parameter `random_state=42` is used to make the results reproducible. This split allows us to train the model on one part of the data and evaluate its performance on unseen data.

```
In [59]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [60]: print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (242, 19)
X_test shape: (61, 19)
y_train shape: (242,)
y_test shape: (61,)
```

## Split Confirmation

The printed shapes confirm that the dataset was successfully divided into training and testing sets. The training set contains 80% of the data, while the testing set contains the remaining 20%. This ensures that the model can later be evaluated on data that was not used during training.

```
In [61]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

## Scaling Choice

We use `StandardScaler` because the input features are measured on very different scales. For example, `chol` ranges in the hundreds while `oldpeak` is typically between 0 and 6. `MinMaxScaler` is more sensitive to extreme minimum and maximum values, which could compress the majority of observations into a narrow interval. Because of this, we used `StandardScaler`. `StandardScaler` makes the features more comparable by centering them around 0 and scaling them to unit variance, which is helpful for many machine learning models.

## Preventing Data Leakage

Scaling was applied only after the train/test split. The scaler was fitted on `X_train` only and then used to transform both `X_train` and `X_test`. This prevents data leakage, because information from the test set is not used during training.

## 7. Conclusion

In this notebook, the dataset was explored and prepared for machine learning. We examined its structure, visualized important patterns, handled missing values in `ca` and `thal`, applied one-hot encoding to categorical features, created a new interaction feature, and applied scaling correctly without data leakage. As a result, the data is now in a cleaner and more suitable form for modelling in the next stage of the project.